

Sistema de Control de Acceso de Estacionamiento SICAE

Descripción

Desarrollar un sistema distribuido de control de acceso para un estacionamiento privado basado en una arquitectura de microservicios orientada al dominio (Domain-Oriented Microservices), con separación funcional de servicios para la administración de usuarios, vehículos y registros de estacionamiento.

La solución deberá implementar mecanismos de autenticación y autorización mediante JSON Web Tokens (JWT), comunicación entre microservicios y persistencia distribuida utilizando múltiples bases de datos independientes por dominio. Asimismo, la plataforma deberá ser desplegada mediante contenedores, permitiendo la independencia, portabilidad y escalabilidad de cada microservicio dentro del entorno de ejecución.

El sistema deberá permitir el registro y administración de usuarios y vehículos autorizados, la validación de accesos al estacionamiento, el almacenamiento de registros de entrada y salida, el cálculo automático de tiempos de permanencia y costos asociados, así como la consulta del historial de movimientos vehiculares dentro de la plataforma.

El proyecto tendrá como finalidad que los estudiantes apliquen conceptos de arquitectura de microservicios, arquitectura en capas, diseño orientado al dominio, seguridad en aplicaciones distribuidas, integración de servicios y administración de múltiples motores de bases de datos y despliegue de aplicaciones mediante la contenerización.

La arquitectura de microservicios define la estructura global del sistema, incluyendo despliegue, independencia de servicios y bases de datos. Mientras que la arquitectura en capas organiza internamente el código de cada microservicio separando responsabilidades mediante Controller, Service y Repository.

Objetivo General

Diseñar e implementar una solución basada en arquitectura de microservicios, aplicando buenas prácticas de desarrollo, separación de responsabilidades, despliegue distribuido y administración de servicios en entornos Linux virtualizados.

Objetivos Específicos

- Comprender los fundamentos de la arquitectura de microservicios.
- Diseñar microservicios independientes y desacoplados.
- Implementar microservicios utilizando una arquitectura en capas basada en Controller, Service y Repository.
- Aplicar principios de separación de responsabilidades y organización del código.
- Desplegar microservicios y bases de datos en contenedores
- Utilizar herramientas de pruebas de APIs como Postman y/o Hoppscotch.
- Gestionar servicios y contenedores en entornos virtualizados.
- Analizar aspectos básicos de escalabilidad y despliegue distribuido.

Alcance del Proyecto

El proyecto contempla el diseño e implementación de una solución basada en arquitectura de microservicios utilizando diversas tecnologías de desarrollo y despliegue.

El alcance incluye:

- Diseño de una solución distribuida basada en microservicios.
- Desarrollo de microservicios independientes.
- Implementación de una arquitectura en capas basada en Controller, Service y Repository dentro de cada microservicio.
- Contenerización de servicios y bases de datos.
- Despliegue distribuido de contenedores.
- Pruebas funcionales de las APIs mediante herramientas como Postman y/o Hoppscotch.

El proyecto no contempla:

- Desarrollo de clientes web.
- Desarrollo de aplicaciones móviles.
- Desarrollo de aplicaciones de escritorio.
- Implementación de interfaces gráficas de usuario.

Dominios y funcionalidades del sistema

- **Servicios de Autenticación (AuthService)**
 - **Login:** Permite autenticar usuarios mediante credenciales válidas (usuario y contraseña) para obtener acceso al sistema.
 - **Reglas de negocio:**
 - Solo usuarios registrados y activos pueden iniciar sesión.
 - Datos obligatorios: usuario y contraseña.
 - Generar token de autenticación.
 - El token deberá de usarse como mecanismo de autenticación para el consumo de los otros servicios.
 - **Validaciones:**
 - Validar los datos obligatorios.
 - Verificar existencia del usuario en la base de datos y verificar que la contraseña sea correcta.
 - Validar el tamaño de los campos
 - **Respuesta:**
 - Datos (idUserio, idRol, rol, usuario, nombre completo, idTipoUsuario y tipo de usuario) del usuario y el token.
- **Servicios de usuario (UserService)**
 - **Registrar:** Permite registrar nuevos usuarios en el sistema
 - **Reglas de negocio:**
 - Solo los usuarios con rol de administrador pueden registrar usuarios.

- El usuario deberá autenticarse previamente mediante el servicio de login y proporcionar un token de autenticación válido para realizar la operación.
 - No deben existir usuario con el mismo correo ni username,
 - Cifrar contraseñas con bcrypt.
 - Usuario activo por defecto.
 - Datos obligatorios: rol, tipo de usuario, nombre, apellido paterno programa educativo, usuario, contraseña, correo, teléfono y tiempo de registro, clave de usuario.
 - La clave de usuario se debe generar internamente y debe de ser única y seguir un patrón (ejemplo: RGR-254).
 - **Validaciones:**
 - Validar los datos obligatorios.
 - Validar formato de correo.
 - Validar mediante el correo y username que no existan usuario repetidos.
 - Validar el tamaño de los campos
 - **Respuesta:**
 - Mensaje que indique si la operación se realizó correctamente o, en caso contrario, un mensaje que especifique la causa por la cual no pudo completarse
- **Editar:** Permite actualizar información de un usuario existente
- **Reglas de negocio:**
 - El usuario deberá autenticarse previamente mediante el servicio de login y proporcionar un token de autenticación válido para realizar la operación.
 - Datos obligatorios: rol, tipo de usuario, nombre, apellido paterno programa educativo, usuario, contraseña, correo, teléfono y tiempo de actualización.
 - No podrá editar directamente el usuario, contraseña y clave del usuario.
 - **Validaciones:**
 - Validar los datos obligatorios.
 - Validar formato de correo.
 - Validar mediante el correo electrónico que no existan usuarios repetidos.
 - Validar el tamaño de los campos
 - **Respuesta:**
 - Mensaje que indique si la operación se realizó correctamente o, en caso contrario, un mensaje que especifique la causa por la cual no pudo completarse
- **Ver perfil:** Permitirá visualizar la información completa del usuario.
- **Reglas de negocio:**

- El usuario deberá autenticarse previamente mediante el servicio de login y proporcionar un token de autenticación válido para realizar la operación.
 - Datos obligatorios: identificador del usuario (idUsuario)
 - **Validaciones:**
 - Validar los datos obligatorios.
 - **Respuesta:**
 - Datos completos del usuario (Rol, nombre completo, tipo de usuario, programa educativo, usuario, correo, teléfono, estatus, clave de usuario, tiempo de creación y actualización)
- **Cambiar estatus:** Permite actualizar es el estatus del usuario (activo/inactivo)
 - **Reglas de negocio:**
 - El usuario deberá autenticarse previamente mediante el servicio de login y proporcionar un token de autenticación válido para realizar la operación.
 - Solo los usuarios con rol de administrador pueden cambiar el estatus de otro usuario.
 - Datos obligatorios: identificador del usuario (idUsuario) y rol (idRol).
 - **Validaciones:**
 - Validar los datos obligatorios.
 - **Respuesta:**
 - Mensaje que indique si la operación se realizó correctamente o, en caso contrario, un mensaje que especifique la causa por la cual no pudo completarse
- **Servicios de vehículo (VehicleService)**
 - **Buscar vehículos:** Permitirá buscar o cargar solo los vehículos de un usuario en específico.
 - **Reglas de negocio:**
 - El usuario deberá autenticarse previamente mediante el servicio de login y proporcionar un token de autenticación válido para realizar la operación.
 - Datos obligatorios: idUsuario
 - **Validaciones:**
 - Validar los datos obligatorios.
 - **Respuesta:**
 - Conjunto de vehículos asociados al usuario (idUsuario, idVehiculo, idModelo, modelo, idMarca, marca, placa, color, año, descripción y estatus)
 - **Registrar:** Persistirá registrar y asociar vehículos a un usuario registrado
 - **Reglas de negocio:**

- El usuario deberá autenticarse previamente mediante el servicio de login y proporcionar un token de autenticación válido para realizar la operación.
 - No deben existir vehículos con la misma placa,
 - Vehículo activo por defecto.
 - Solo podrá tener 4 vehículos activos al mismo tiempo, en caso de tener mas no deberá dejar registrar otro vehículo hasta que desactive uno.
 - Datos obligatorios: idUsuario, idVehiculo, idModelo, placa, color, año, estatus y descripción.
 - **Validaciones:**
 - Validar los datos obligatorios.
 - Validar que solo tenga 4 vehículos activos al mismo tiempo.
 - Validar mediante la placa que no existan vehículos repetidos.
 - Validar el tamaño de los campos
 - **Respuesta:**
 - Mensaje que indique si la operación se realizó correctamente o, en caso contrario, un mensaje que especifique la causa por la cual no pudo completarse
- **Editar:** Permite actualizar información de un vehículo existente.
 - **Reglas de negocio:**
 - El usuario deberá autenticarse previamente mediante el servicio de login y proporcionar un token de autenticación válido para realizar la operación.
 - Solo se podrá actualizar el vehículo que estén asociados al usuario autenticado actualmente.
 - Datos obligatorios: idUsuario, idVehiculo, idModelo, placa, color, año y descripción.
 - **Validaciones:**
 - Validar los datos obligatorios.
 - Validar mediante la placa que no existan vehículos repetidos.
 - Validar el tamaño de los campos
 - **Respuesta:**
 - Mensaje que indique si la operación se realizó correctamente o, en caso contrario, un mensaje que especifique la causa por la cual no pudo completarse
- **Cambiar estatus:** Permitirá actualizar es el estatus del vehículo (activo/inactivo)
 - **Reglas de negocio:**
 - El usuario deberá autenticarse previamente mediante el servicio de login y proporcionar un token de autenticación válido para realizar la operación.
 - Solo se podrá actualizar el estatus de los vehículos que estén asociados al usuario autenticado actualmente.

- Datos obligatorios: identificador del usuario (idUsuraio) y del vehículo (idVehiculo)
 - **Validaciones:**
 - Validar los datos obligatorios.
 - **Respuesta:**
 - Mensaje que indique si la operación se realizó correctamente o, en caso contrario, un mensaje que especifique la causa por la cual no pudo completarse
- **Servicios de estacionamiento (ParkingService)**
 - **Registrar movimiento (entrada):** Permite registrar la entrada de los vehículos al estacionamiento.
 - **Reglas de negocio:**
 - Solo podrán entrar a vehículos que cumplan con lo siguiente:
 - Estén asociados a un usuario registrado y activo.
 - Solo podrán acceder máximo de 2 vehículos por usuario y los vehículos deben de estar activos.
 - Datos obligatorios: claveUsuario, placa, tiempo de entrada, tiempo creación, tarifa, espacio (idEspacio).
 - Si se cumplen las validaciones, una vez creado el registro de la entrada se deberá actualizar el campo “ocupado” del espacio asignado al auto.
 - **Validaciones:**
 - Validar que el usuario este activo, Validar los datos obligatorios.
 - Validar la asociación entre usuario y vehículo, que el vehículo que pretende entrar pertenezca al usuario que pretende ingresar el auto.
 - **Respuesta:**
 - Si la operación de entrada se realizó correctamente deberá devolver los datos del movimiento (idMovimiento, tiempo de Entrada, espacio asignado, tarifa por Hora), en caso contrario, un mensaje que especifique la causa por la cual no pudo ingresar.
 - **Actualizar movimiento (salida):** Permite registrar la salida de los vehículos del estacionamiento.
 - **Reglas de negocio:**
 - Solo podrán salir los vehículos que cumplan con lo siguiente:
 - Estén asociados a un usuario registrado y activo.
 - Datos obligatorios: claveUsuario, placa, tiempo de salida, tiempo actualización, costo total, horas cobradas, minutos Estacionado.
 - Si se cumplen las validaciones, una vez actualizado el registro (salida) se deberá actualizar el campo “ocupado” del espacio que está liberando el auto.

- **Validaciones:**
 - Validar que el usuario este activo, Validar los datos obligatorios.
 - Validar la asociación entre usuario y vehículo, que el vehículo que pretende salir pertenezca al usuario que pretende sacar el auto.
 - **Respuesta:**
 - Si la operación de salida se realizó correctamente deberá devolver los datos del movimiento (idMovimiento, tiempo Entrada, tiempo de salida espacio asignado, tarifa Hora, costo total, horas cobradas), en caso contrario, un mensaje que especifique la causa por la cual no puedo salir.
- **Consultar espacios:** Permite consultar los espacios del estacionamiento.
- **Reglas de negocio:**
 - El usuario deberá autenticarse previamente mediante el servicio de login y proporcionar un token de autenticación válido para realizar la operación.
 - **Respuesta:**
 - Datos de los espacios disponibles.

Reglas de negocio generales (aplican para todos los microservicios)

- Para consumir cualquier servicio, el usuario deberá autenticarse previamente mediante el servicio de login y proporcionar un token de autenticación válido en cada solicitud.
- Toda solicitud deberá validar datos obligatorios, formatos y restricciones de negocio.
- No se permitirán registros duplicados.
- Todos los servicios deberán retornar mensajes claros de éxito o error.
- Cada microservicio deberá implementar una arquitectura en capas basada en Controller, Service y Repository.
- Cada microservicio gestionará de forma independiente su lógica de negocio y base de datos.
- Las entidades principales no deberán eliminarse físicamente, únicamente cambiar su estatus.
- El sistema deberá garantizar la integridad y consistencia de la información.

Tecnologías de desarrollo

Para la implementación de la solución, deberán desarrollar la arquitectura de microservicios utilizando java o si el alumno lo prefiere puede utilizar dos tecnologías backend diferentes, con el propósito de aplicar interoperabilidad entre servicios y comunicación distribuida en entornos heterogéneos.

Backend Java

- Java 17 o superior
- RestController (springframework.web)
- Maven

- MyBatis
- Apache Tomcat
- NetBeans

Los microservicios se deberán desarrollar bajo una arquitectura en capas basada en Controller, Service y Repository

Persistencia y distribución de bases de datos

La solución deberá implementar persistencia distribuida, asignando una base de datos independiente por dominio funcional, siguiendo el principio de desacoplamiento de microservicios.

PostgreSQL

Base de datos correspondiente al dominio de usuarios:

- **usuarios**
- roles (catálogo)
- tipos de usuario (catálogo)
- programas educativos (catálogo)

MySQL

Base de datos correspondiente al dominio de vehículos:

- marcas (catálogo)
- modelos (catálogo)
- **vehículos**

Base de datos correspondiente al dominio de estacionamiento:

- cajones de estacionamiento (catálogo)
- **movimientos de estacionamiento**

Cada microservicio deberá administrar exclusivamente su propia base de datos, evitando dependencias directas y relaciones físicas entre dominios.

Se les entregarán 3 scripts que contendrán los esquemas y datos para cada base de datos, solo tendrán que importar las bases de datos.

Seguridad

La solución deberá implementar mecanismos de autenticación y autorización para el acceso seguro a los servicios.

Tecnologías requeridas

- JWT (JSON Web Token) para autenticación y autorización.
- Bcrypt para el cifrado y almacenamiento seguro de contraseñas.

Comunicación

La comunicación con los microservicios deberá realizarse mediante una APIs REST.

Protocolos y formatos

- REST API para exposición y consumo de servicios
- JSON como formato estándar para el intercambio de datos entre microservicios

Los servicios deberán permitir la comunicación de manera desacoplada, permitiendo interoperabilidad entre los endpoints.

Despliegue

La solución deberá ser desplegada utilizando contenedores, garantizando portabilidad, aislamiento y facilidad de despliegue de cada microservicio y sus respectivas bases de datos.

- **Servidor1:** deberá tener lo siguiente
 - **Contenedor1:** Microservicios de autenticación
 - **Contenedor2:** Microservicios de usuario
 - **Contenedor3:** Base de datos de usuarios
- **Servidor2:** deberá tener lo siguiente
 - **Contenedor1:** Microservicios de vehículo
 - **Contenedor3:** Base de datos de vehículo
- **Servidor3:** deberá tener lo siguiente
 - **Contenedor1:** Microservicios de estacionamiento
 - **Contenedor2:** Base de datos de estacionamiento

Entregables

Cada equipo deberá entregar:

1. Código fuente de los 3 proyectos de microservicios y sus bases de datos.
2. Documento técnico del proyecto (se les proporcionará una platilla con lo que deberá de contener el documento).
3. Colección de Postman
4. Archivos de configuración, DockerFile de los contenedores, Scripts, etc.

Fecha de entrega

- La presentación y demostración del proyecto por parte del equipo se realizará tres días después de la aplicación del examen ordinario. No se otorgarán prórrogas ni cambios de fecha.
- La documentación técnica, deberán entregarse el mismo día en que se presente el examen ordinario. Esto tiene como finalidad que el profesor pueda revisar previamente la documentación y validar el funcionamiento de la solución durante la presentación del proyecto. Deberá subir una carpeta comprimida a Eminus4, la carpeta deberá seguir la siguiente nomenclatura: **Equipo_NumEquipo.zip**, por ejemplo **Equipo_3.zip**.

Forma de evaluación y consideraciones

- La evaluación del proyecto se realizará considerando tanto la documentación entregada como la presentación y demostración funcional de la solución desarrollada por los equipos.
- Durante la revisión se verificará que el proyecto cumpla correctamente con todos los requerimientos establecidos y con los criterios definidos en la rúbrica de evaluación.
- En caso de que el proyecto no cumpla con los requisitos mínimos solicitados o no alcance los criterios de evaluación establecidos, este no será considerado como aprobado y deberá ser entregado nuevamente en periodo extraordinario o en título, según corresponda.
- El proyecto constituye un requisito obligatorio para acreditar la experiencia educativa, por lo que su entrega y funcionamiento son indispensables.
- El proyecto deberá funcionar correctamente (100%) durante la demostración.
- Se evaluará la organización del código y la claridad de la documentación.
- Los alumnos deberán explicar el funcionamiento de la solución completa.
- Se permite el uso de documentación oficial y recursos de apoyo.
- Queda estrictamente **prohibido el uso de Inteligencias artificiales generativas para el desarrollo del proyecto**, en caso de detectar que el código fuente fue desarrollado por una IA se cancelará el proyecto y no será tomado en cuenta para la evaluación.

Consideración Importante

- Todos los integrantes del equipo deberán participar activamente tanto en las actividades de documentación como en la implementación (codificación), configuración y pruebas del proyecto.
- Uno de los objetivos principales fortalece la lógica de programación, la integración de tecnologías y el diseño de sistemas distribuidos similares a los utilizados en entornos profesionales de desarrollo de software.
- En caso de detectarse que alguno de los integrantes no participó en el desarrollo técnico del proyecto, este podrá no ser considerado para evaluación.